

Relational Design and Normalization

Course notes - Department of Computer Science

1. The relational model

A relation is a set of tuples over a fixed set of attributes. Because it is a set, there is no inherent ordering of rows and no duplicates. Each attribute draws values from a domain, and the special value NULL marks information that is absent or inapplicable.

A candidate key is a minimal set of attributes that uniquely identifies a tuple. One candidate key is designated the primary key; the others remain alternate keys and are usually enforced with unique constraints.

2. Functional dependencies

An attribute set Y is functionally dependent on X, written $X \rightarrow Y$, when any two tuples agreeing on X must agree on Y. Functional dependencies encode the business rules of the domain and are the raw material of normalization.

Armstrong's axioms - reflexivity, augmentation and transitivity - let us derive every dependency implied by a given set, and computing the closure of an attribute set tells us whether it is a superkey.

3. Update anomalies

A poorly factored table suffers three characteristic problems. An insertion anomaly prevents recording a fact because unrelated information is missing. A deletion anomaly loses a fact as a side effect of removing something else. An update anomaly requires the same fact to be changed in many rows, and leaves the table inconsistent if any are missed.

Normalization removes these anomalies by splitting relations so that every fact is stored exactly once.

4. The normal forms

First normal form requires every attribute to be atomic, ruling out repeating groups and nested structures. Second normal form additionally forbids partial dependencies, where a non-key attribute depends on only part of a composite key.

Third normal form forbids transitive dependencies, where a non-key attribute depends on another non-key attribute. Boyce-Codd normal form tightens this: every non-trivial determinant must be a superkey. Higher forms address multi-valued and join dependencies and are rarely needed in practice.

5. When to denormalize

Normalization optimises for correctness under update. Read-heavy workloads sometimes pay for that in join cost, and a controlled denormalization - a materialised aggregate, a duplicated lookup column - can be the right engineering trade.

Denormalize deliberately, after measuring, and document which copy is authoritative. An undocumented duplicate is simply a bug waiting to surface.